



Jour 2 — Création et gestion de DAGs

Objectif : Maîtriser DAGs, opérateurs, dépendances, XCom, sensors

🎯 **Livrable** : Pipeline ETL complet planifié



DAGs



Dépendances



XCom

Objectifs & Agenda - Jour 2

Programme détaillé pour la maîtrise des DAGs et opérateurs :

Objectif 1 - Structure et Planification

Comprendre l'architecture interne d'un DAG et maîtriser les expressions cron et timedelta.

Objectif 2 - Opérateurs et Dépendances

Utiliser Python/Bash/Branch Operators et gérer des flux d'exécution complexes.

Objectif 3 - Communication Inter-Tâches

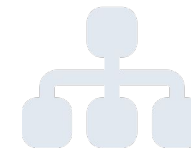
Maîtriser XCom pour échanger des données et des métadonnées entre les tâches.

Objectif 4 - Intégration Externe

Mettre en place des Hooks pour les connexions et des Sensors pour l'attente conditionnelle.

Livrable - Pipeline ETL Complet

Créer un pipeline Extract-Transform-Load automatisé et robuste.



Structure approfondie d'un DAG

Paramètres essentiels (Identité & Exécution)

-  **dag_id** : Identifiant unique global du DAG
-  **description / tags** : Métadonnées pour l'interface UI et le filtrage
-  **start_date / schedule_interval** : Date de début et fréquence d'exécution
-  **catchup** : Booléen (True/False) pour le rattrapage des runs passés



Contrôle & Documentation avancée

-  **max_active_runs** : Limite de runs simultanés pour ce DAG
-  **dagrun_timeout** : Temps maximum global autorisé pour une exécution complète
-  **default_args** : Dictionnaire de paramètres partagés par toutes les tâches (retries, owner...)
-  **doc_md** : Documentation au format Markdown, visible directement dans l'interface web

Planification: presets et timedelta

Les Presets standards

-  `@once` : Exécution unique au démarrage
-  `@hourly` : Chaque heure à la minute 0
-  `@daily` : Chaque jour à minuit (00:00)
-  `@weekly` : Chaque dimanche à minuit
-  `@monthly` / `@yearly` : Premier jour du mois/année

Cron & Timedelta

 **Expressions Cron :**
Format classique : `minute heure jour mois jourSemaine`
Permet une planification très fine.

 **Timedelta (Python) :**
Planification relative. Exemple : `timedelta(hours=2)`
S'exécute toutes les 2 heures après le `start_date`.

 **Déclenchement Manuel :**
Utiliser `schedule_interval=None` pour des DAGs déclenchés manuellement ou via API.

Expressions Cron : Exemples

Comprendre et utiliser les expressions Cron personnalisées pour planifier vos DAGs avec précision.

Expression Cron	Description	Cas d'usage courant
0 6 * * 1-5	Tous les jours de la semaine (du lundi au vendredi) à 06:00.	Préparation des rapports matinaux pour les équipes métier.
*/15 * * * *	Toutes les 15 minutes, 24h/24, 7j/7.	Ingestion de données quasi temps réel, synchronisation rapide.
0 0,12 * * *	Deux fois par jour : à minuit (00:00) et à midi (12:00).	Mises à jour biquotidiennes de bases de données.
None	Aucune planification automatique (nécessite l'utilisation du mot-clé None en Python).	DAGs déclenchés uniquement manuellement ou via une API externe.

Opérateurs : Python et Bash

Les fondations de vos tâches



PythonOperator exécute une fonction Python classique. Utilisez `python_callable` pour référencer la fonction et `op_kwargs` pour passer des arguments.



BashOperator exécute des commandes système. Parfait pour lancer des scripts existants ou des commandes utilitaires via `bash_command`.



XCom natif : Tout ce qui est retourné par la fonction Python (`return valeur`) est automatiquement poussé dans XCom sous la clé `return_value`.

```
operators_example.py

from airflow.operators.python import PythonOperator
from airflow.operators.bash import BashOperator

def saluer(nom, **kwargs):
    # ds est la date d'exécution injectée par Airflow
    print(f"Bonjour {nom} ! Traitement du {kwargs['ds']}")
    return "Succès" # Poussé auto dans XCom

# 1. Utilisation du PythonOperator
tache_python = PythonOperator(
    task_id='dire_bonjour',
    python_callable=saluer,
    op_kwargs={'nom': 'Airflow'},
)

# 2. Utilisation du BashOperator avec variables d'environnement
tache_bash = BashOperator(
    task_id='script_externe',
    bash_command='/opt/scripts/traitement.sh ', # Espace final !
    env={'EXECUTION_DATE': '{{ ds }}'},
)
```

Branching et Trigger Rules

BranchPythonOperator

-  Permet de créer des branches conditionnelles dans le flux d'exécution.
-  Doit retourner le **task_id** (ou une liste de task_ids) à exécuter ensuite.
-  Les branches non retournées sont ignorées (état "skipped").
-  Utilisez **EmptyOperator** (ex-DummyOperator) comme point de convergence.

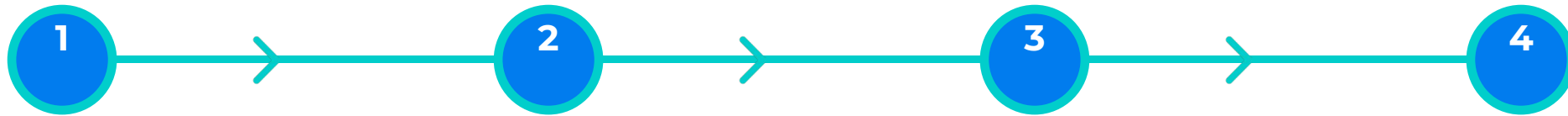


Trigger Rules

-  **ALL_SUCCESS** (défaut) : Tous les parents ont réussi.
-  **ONE_SUCCESS** : S'exécute dès qu'un parent réussit.
-  **ALL_DONE** : Exécute peu importe l'état des parents (réussite, échec, ignoré).
-  **NONE_FAILED** : Aucun parent n'a échoué (peuvent être ignorés).
-  **ALL_FAILED** : Tous les parents ont échoué.

Dépendances avancées

Techniques pour orchestrer des workflows complexes et lisibles :



Opérateurs de base

La méthode classique et intuitive :

```
task_a >> task_b
```

```
task_b << task_a
```

Listes (Fan-in/out)

Parallélisation et convergence :

```
task_a >> [task_b, task_c]
```

```
[task_b, task_c] >> task_d
```

Fonction chain()

Pour des séquences longues ou des dépendances croisées :

```
chain(a, [b, c], d)
```

Équivaut à : `a>>b>>d` ET `a>>c>>d`

Bonnes pratiques

Maintenir un graphe lisible (vue Graph)

Gérer proprement le fan-in/fan-out

Utiliser des EmptyOperators comme points de convergence

XCom : Communication inter-tâches

Principes fondamentaux



Cross-Communication (XCom) : permet l'échange de petites données (métadonnées) entre les tâches.



Les données XCom sont stockées dans la **Metadata DB** d'Airflow (Postgres/MySQL).



xcom_push : envoie explicitement une valeur (clé, valeur) ou via un simple ``return`` (clé 'return_value').



xcom_pull : récupère une valeur en ciblant le ``task_id`` source et la clé.



Limites et Bonnes Pratiques



Taille limitée : XCom n'est pas fait pour passer des DataFrames ou de gros volumes (limite DB, ex: Postgres 1GB, MySQL 64KB).



Le bon pattern : utilisez XCom pour passer des références (chemins de fichiers, IDs).



Stockage externe : pour les vraies données, stockez sur S3/GCS/HDFS et passez l'URI via XCom.



Sécurité : ne stockez **jamais** de mots de passe ou secrets en clair dans XCom (lisibles en UI).

XCom : Exemples Rapides

Comment utiliser XCom

- 1 Pousser des données :**
T1 pousse une valeur, soit explicitement via `ti.xcom_push`, soit par la valeur de retour (clé automatique `'return_value'`).
- 2 Récupérer des données :**
T2 lit la valeur via `ti.xcom_pull` en spécifiant l'ID de la tâche source et éventuellement la clé.
- 3 Templates Jinja :**
Utilisation dans les paramètres supportant le templating, par exemple `{{ ti.xcom_pull(task_ids='T1') }}`

```
python - xcom_example.py

def produire_donnees(**kwargs):
    ti = kwargs['ti']
    # Méthode explicite
    ti.xcom_push(key='ma_cle', value={'data': [1, 2, 3]})
    # Méthode via valeur de retour
    return "Valeur retournée"

def consommer_donnees(**kwargs):
    ti = kwargs['ti']
    # Récupérer par clé
    donnees = ti.xcom_pull(task_ids='produire_donnees', key='ma_cle')
    # Récupérer la valeur de retour (return_value)
    retour = ti.xcom_pull(task_ids='produire_donnees')
    print(f"Données: {donnees}, Retour: {retour}")

# Utilisation dans Jinja (BashOperator)
task_template = BashOperator(
    task_id='utiliser_xcom',
    bash_command='echo "Valeur: {{ ti.xcom_pull(task_ids=\"produire\") }}"',
)
```

Hooks: connexions réutilisables

Concept des Hooks



Les Hooks **encapsulent la logique** de connexion aux systèmes externes (DB, API, Cloud).



Ils utilisent les **Connections Airflow** pour ne jamais hardcoder de credentials dans le code.



PostgresHook: `get_records()`, `get_pandas_df()`, `insert_rows()`.



HttpHook: `run()` pour exécuter des requêtes GET/POST avec gestion automatique des headers.



Bonne Pratique : Définissez toujours les connexions via l'UI d'Airflow ou via les variables d'environnement.

```
python - utilisation des hooks

from airflow.providers.postgres.hooks.postgres import PostgresHook
from airflow.providers.http.hooks.http import HttpHook

# Exemple 1 : Utilisation de PostgresHook
def extraire_et_traiter(**kwargs):
    # Instanciation avec l'ID de connexion défini dans l'UI
    hook = PostgresHook(postgres_conn_id='ma_db_postgres')

    # Exécution de requête et récupération sous forme de DataFrame
    df = hook.get_pandas_df("""
        SELECT produit, SUM(montant) as total
        FROM ventes WHERE date_vente >= '{{ ds }}'
        GROUP BY produit
    """)

    # Insertion de données simplifiée
    hook.insert_rows(
        table='resultats',
        rows=[('Produit A', 150.00)],
        target_fields=['produit', 'montant']
    )

# Exemple 2 : Utilisation de HttpHook
def appeler_api(**kwargs):
    hook = HttpHook(method='GET', http_conn_id='mon_api')
    response = hook.run(
        endpoint='/search',
        data={'query': 'airflow'},
        headers={'Accept': 'application/json'}
    )
    return response.json()
```

Sensors : attendre une condition

Les Sensors en Airflow



FileSensor : Attend l'apparition d'un fichier ou répertoire spécifique (ex: filepath).



ExternalTaskSensor : Synchronise les exécutions en attendant la fin d'une tâche dans un autre DAG.



HttpSensor : Interroge un endpoint périodiquement et valide avec un `response_check`.



Le mode **reschedule** libère le worker entre les vérifications, idéal pour de longues attentes (poke = actif en continu).

python - sensors.py

```
from airflow.sensors.filesystem import FileSensor
from airflow.sensors.external_task import ExternalTaskSensor
from airflow.providers.http.sensors.http import HttpSensor

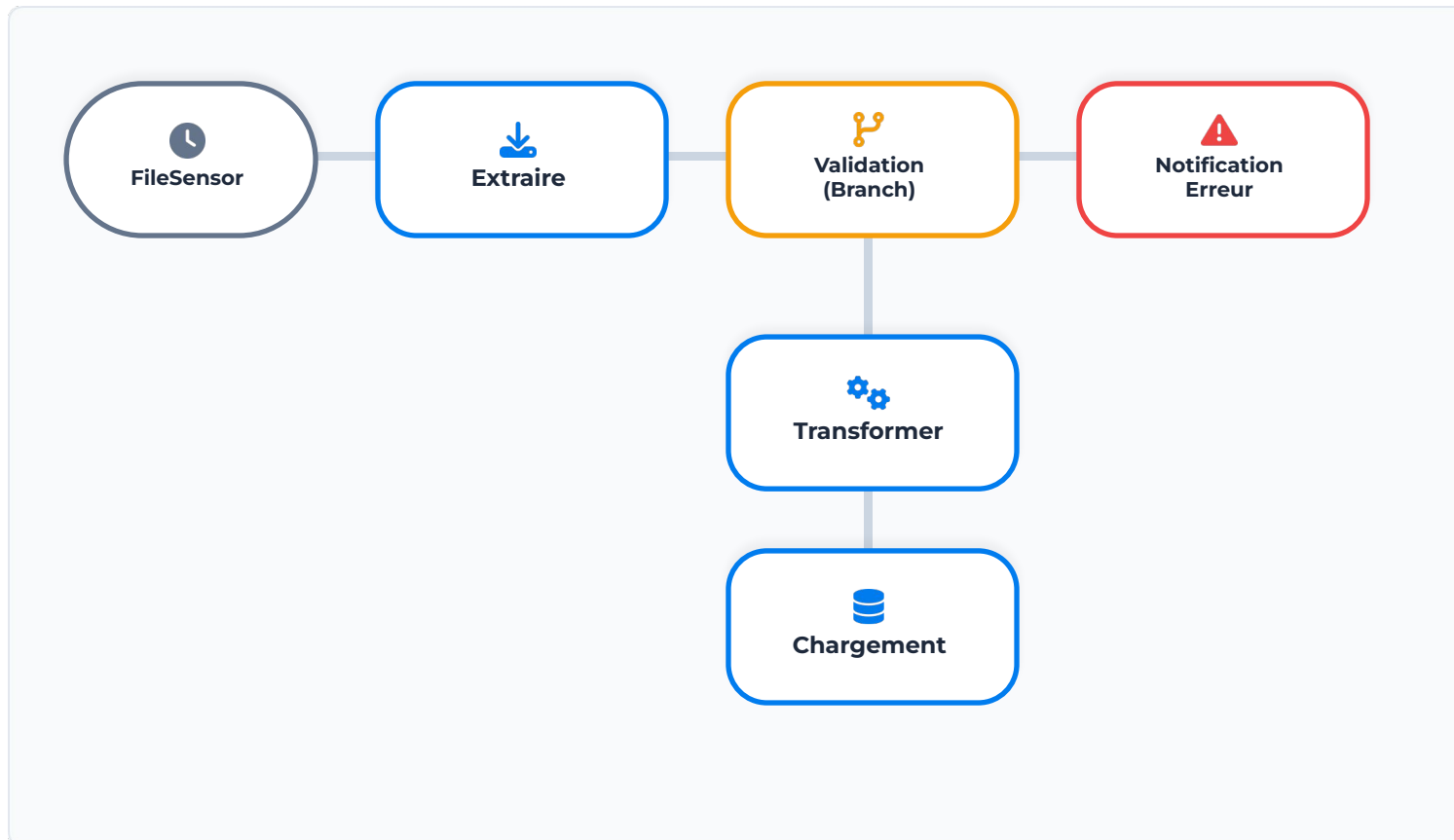
# Attendre qu'un fichier soit déposé (ex: FTP/Drop)
attendre_fichier = FileSensor(
    task_id='attendre_fichier',
    filepath='/data/input/fichier.csv',
    poke_interval=60, # Vérifie toutes les 60s
    timeout=3600,     # Timeout après 1 heure
    mode='reschedule' # Libère le worker
)

# Attendre la complétion d'un autre DAG
attendre_dag = ExternalTaskSensor(
    task_id='attendre_autre_dag',
    external_dag_id='autre_dag_important',
    external_task_id='derniere_tache'
)

# Attendre la disponibilité d'une API externe
attendre_api = HttpSensor(
    task_id='attendre_api',
    http_conn_id='mon_api_connexion',
    endpoint='/health',
    response_check=lambda response: response.status_code == 200
)
```

Pipeline ETL — Design

Conception d'un workflow robuste avec branchement conditionnel



▶ Déclenchement

FileSensor: Mode reschedule pour ne pas bloquer les workers en attendant le fichier.

🔗 Branchement

BranchPythonOperator: Décide du chemin selon la qualité des données (valides ou erreur).

⚙️ Traitement

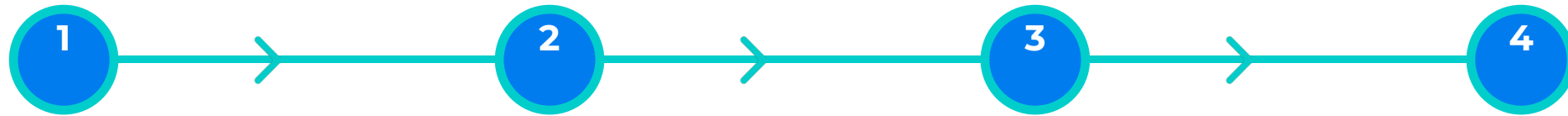
Extraction/Transformation: Via Python ou scripts externes.

Chargement: DB ou Data Lake.

Notifications: Slack/Email en fin ou sur erreur.

Pipeline ETL — Étapes

Décomposition du flux de données pour un pipeline ETL standard orchestré par Airflow :



Attendre source

Patienter jusqu'à la disponibilité des données :

Utilisation de Sensors (ex: FileSensor, S3KeySensor)

Mode `reschedule` pour économiser les workers

Extraire

Récupérer les données depuis la source :

Hooks vers Base de données ou API externes

Validation de la qualité (schéma, volume)

Transformer

Appliquer la logique métier aux données :

PythonOperator (Pandas) pour petits volumes

SparkSubmitOperator pour du Big Data

Charger

Sauvegarder le résultat final :

Insertion en Base (Postgres, MySQL)

Dépôt sur Data Lake (S3) ou HDFS

TP Jour 2 : Pipeline ETL et XCom

Mise en pratique : Branchements, capteurs et communication inter-tâches

Consignes de l'exercice

- 1 **DAG avec branchement** : Générez un nombre aléatoire. S'il est pair, exécutez la tâche "pair". S'il est impair, exécutez "impair". Convergez ensuite vers une tâche finale.
- 2 **Communication XCom** : Créez un second DAG avec :
 - Tâche 1 : Génère une liste de 5 nombres et la retourne
 - Tâche 2 : Récupère la liste via XCom et calcule sa somme
 - Tâche 3 : Affiche le résultat final
- 3 **Sensor personnalisé** : Créez un DAG qui attend que le fichier `/tmp/go.txt` existe avant de déclencher la suite du workflow.

Objectif Bonus

- ✓ Pour le DAG avec branchement, assurez-vous que la tâche finale s'exécute toujours, quelle que soit la branche empruntée.
- ✓ Indice : Utilisez `TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS` sur la tâche finale.

Conseils & Astuces

- La valeur retournée par la fonction d'un `PythonOperator` est automatiquement poussée dans XCom.
- Pour le capteur, utilisez `mode='reschedule'` au lieu du mode par défaut `'poke'` pour ne pas bloquer un worker indéfiniment pendant l'attente du fichier.

Checkpoint Jour 2

Testez vos connaissances sur les concepts clés abordés aujourd'hui :

1

Différence Operator vs Task Instance ?

?

2

Quand utiliser BranchPythonOperator ?

?

3

Bon usage de XCom ?

?

4

Mode reschedule : avantages ?

?

Récap — Jour 2

L'essentiel sur la création et la gestion avancée des DAGs :



Création de DAGs complets avec planification (cron/timedelta)



Maîtrise des **Operators clés** et de la gestion des **dépendances avancées**



Utilisation de **XCom** pour la communication, **Hooks** et **Sensors**



Résultat : Vous êtes capables de concevoir un pipeline ETL complet, prêt à être adapté à vos cas d'usage réels.